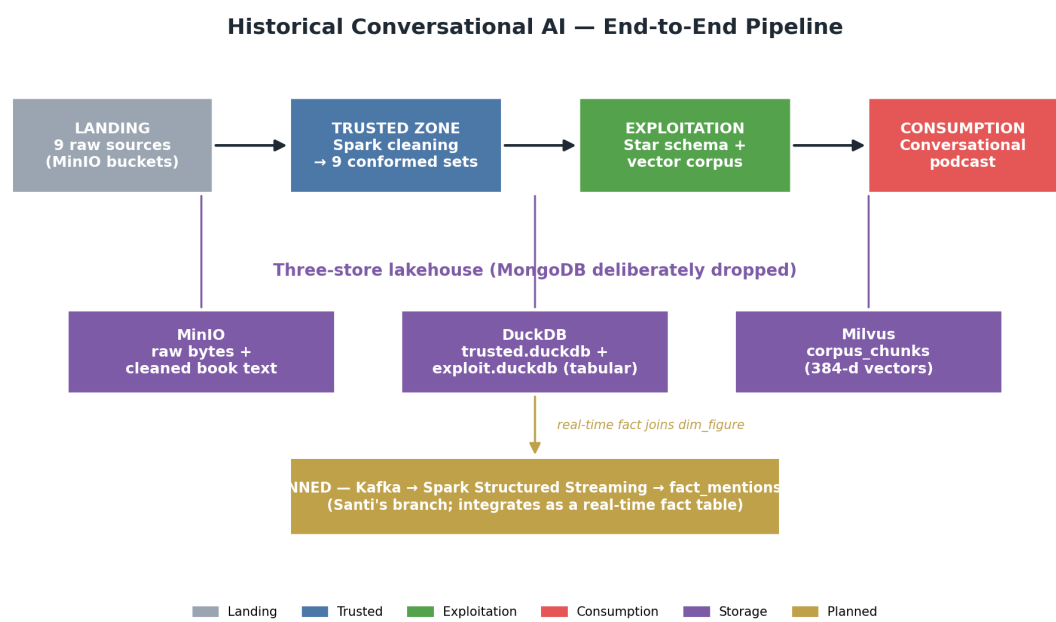


Historical Conversational AI

BDM Project 2 — Technical Report
Exploitation & Consumption Zones · pipeline logic and design



Author: Albert Puiggròs · Status: pipeline green end-to-end · Streaming: planned integration

1 · What we built

The product is a **generated conversational podcast**: for any of nine historical figures (5 philosophers — Plato, Descartes, Kant, Hegel, Nietzsche; 2 scientists — Einstein, Darwin; 2 authors — Wilde, Twain) the system produces a host-interview episode where the figure answers *in character*, grounded in what they actually wrote, in a voice blended from their own maxims and prose, on an intellectually rich topic loosely inspired by recent news.

Every architectural choice is forced by a downstream consumer — the grading criterion the course rewards. The pipeline is a **three-store lakehouse**: MinIO (raw bytes + cleaned book text), DuckDB (all tabular data — Trusted and Exploitation), Milvus (sentence-embedding vectors). MongoDB was deliberately dropped after the Trusted Zone proved DuckDB handles the semi-structured sources cleanly — adding it would have been a tool with no consumer to justify it.

Historical Conversational AI — End-to-End Pipeline

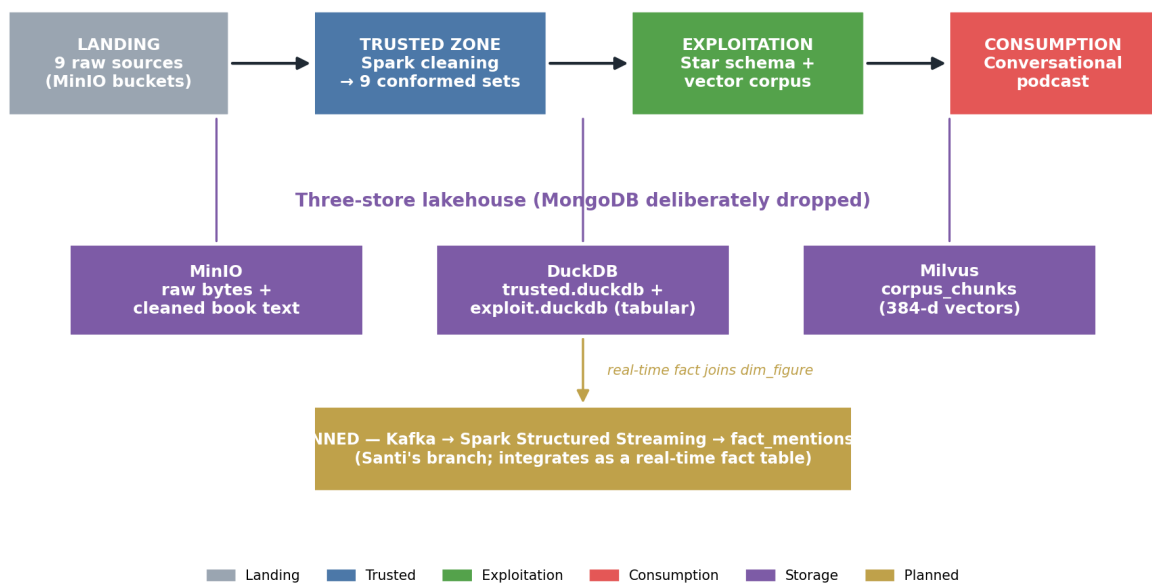


Figure 1 — The four zones and the three stores. Streaming (bottom) is designed and pending integration with Santi's branch.

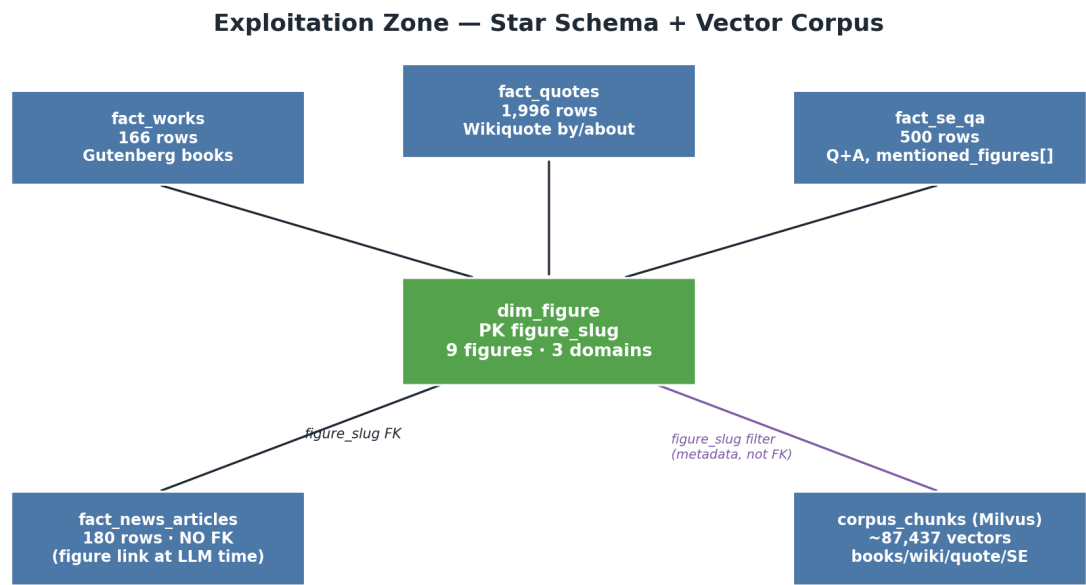
2 · Trusted Zone (recap)

The Trusted Zone is upstream of this report's focus, so only its contract matters here: nine heterogeneous sources are ingested to MinIO and cleaned by Spark jobs into conformed, deduplicated, type-safe sets. Spark is justified here — it reads JSON/text from MinIO in parallel at volume. The outputs land as **trusted_*** tables in `trusted.duckdb` plus boilerplate-stripped book text back in MinIO. These are the only inputs the Exploitation Zone consumes.

Source	Trusted output	Feeds (Exploitation)
Philosophers API	trusted_philosophers	dim_figure (bio)
Wikipedia	trusted_wikipedia	dim_figure (summary/desc)
Wikiquote	trusted_wikiquote	fact_quotes, corpus_chunks
Gutenberg catalog	trusted_gutenberg_*	fact_works
Gutenberg books	cleaned text in MinIO	corpus_chunks
GNews	trusted_news_articles	fact_news_articles
Philosophy Stack Exch.	trusted_se_*	fact_se_qa, corpus
Figure images	trusted_philosopher_images	dim_figure
Mentions stream	(Santi — planned)	fact_mentions_1m

3 · Exploitation Zone

The Exploitation Zone reshapes the trusted sets into a **star schema** plus a **vector corpus**, each table purpose-built for a named consumer (dashboard KPIs or the podcast agents). All tabular work is pure DuckDB SQL — source and target are both DuckDB and the joins are tiny, so a Spark JVM would add startup cost with zero throughput gain. Spark is used in exactly one place: the embedding job (§3.3).



fact_news_articles carries no figure_slug by design — GNews is category-broad; the Interviewer reasons figure↔news at prompt time.

Figure 2 — dim_figure is the single conformed dimension; every fact joins it on figure_slug, except news (by design) and the Milvus corpus (slug is a metadata filter, not a SQL FK).

3.1 · The conformed dimension

dim_figure anchors on the 9-figure character registry and LEFT JOINS the trusted sets, so the 5 philosophers get school/dates/interests while scientists and authors simply carry NULL there — one honest row per figure. It also carries the Wikipedia *summary* and one-line *description*, the factual anchor every agent reads.

3.2 · The fact tables (derivation logic)

Table	Grain	Key derivation logic
fact_works	book × figure	INNER JOIN dim_figure; keeps all books, has_local_text flags the embeddable subset
fact_quotes	quote	quote_type by_figure / about_figure — by_figure feeds Voice, about_figure feeds Reasoner
fact_news_articles	article	Pass-through, NO figure_slug — GNews is category-broad; figure relevance decided at LLM
fact_se_qa	question	Top answer inlined (accepted > score); mentioned_figures[] via tag-prefix + word-boundary

Live counts: 9 figures (5 philosophy / 2 science / 2 literature), 166 works, 1,996 quotes, 180 news articles, 500 Stack-Exchange Q&A.

3.3 · corpus_chunks — the Spark scalability proof

The embedding job is the project's deliberate Spark scalability demonstration. It reads ~162 books from MinIO, windows each into 220-word chunks with 40-word overlap, embeds them with all-MiniLM-L6-v2

(384-d, normalized for COSINE) and writes them to the Milvus corpus_chunks collection (HNSW, M=16, efConstruction=200), with the chunk text inlined for single-call retrieval. The verified collection holds **≈87,437 vectors** across books, Wikipedia, Wikiquote and figure-linked Stack Exchange.

The scaling lesson is in the execution model: Spark parallelizes only the embarrassingly-parallel MiniO read+chunk; embedding and insertion are then *streamed to the driver* via toLocalIterator() in 1,000-row micro-batches with a single model load — constant memory regardless of corpus size. A naive collect() with a per-partition model is what does not scale, and showing that contrast is the point.

4 · Consumption Zone — the product

Four cooperating agents turn the Exploitation stores into a single-figure interview episode. Generation is injected as a provider-swappable `llm_fn` (Anthropic Claude); with no key the pipeline still emits the full episode structure, so it is testable offline.

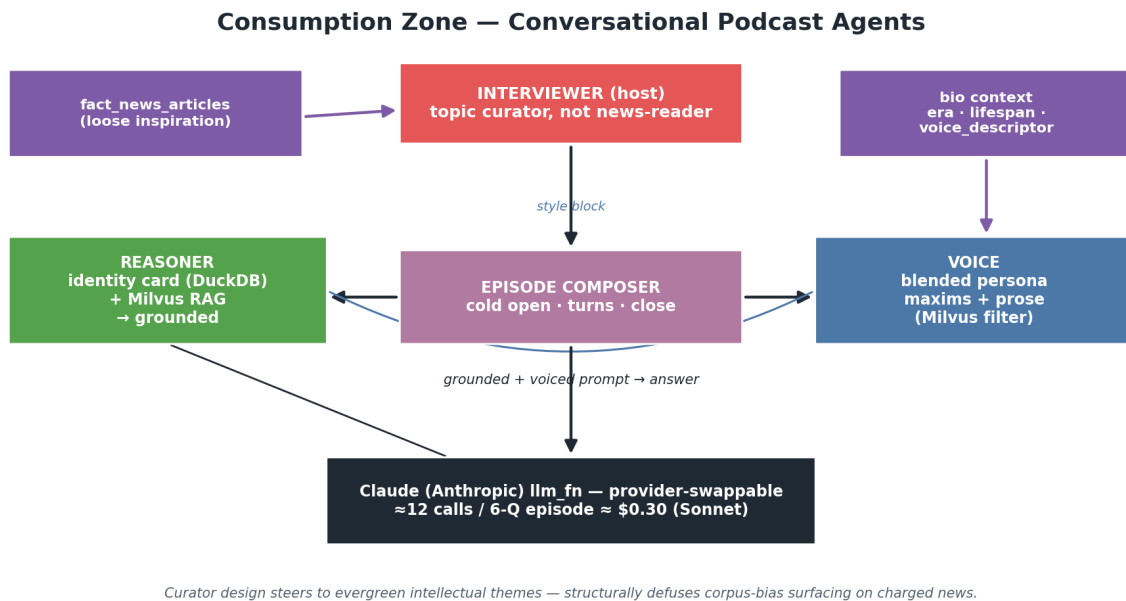


Figure 3 — Interviewer frames; Composer drives the loop; Reasoner grounds; Voice styles. Facts and style stay strictly separated.

4.1 · Reasoner — substance

Loads the DuckDB identity card and retrieves the top-k figure-filtered passages from Milvus, then assembles one grounded prompt that forbids invention beyond the evidence. If Milvus is unreachable it degrades to identity-card-only rather than failing.

4.2 · Voice — style (blended)

Reuses the Reasoner's retrieval twice: the figure's own Wikiquote maxims (cadence) and one Gutenberg prose excerpt (register), both queried with the interview question so the exemplars are topically adjacent. The result is injected as a style block — it changes how the answer *sounds*, never what it asserts.

4.3 · Interviewer — a topic curator

Crucially *not* a news-reader. RAG over complete historical corpora will surface period bigotry if charged news is matched against it (an observed failure). The Interviewer instead curates evergreen, intellectually rich themes (AI, scientific truth, social behaviour, ethics), using headlines only as loose inspiration when they genuinely fit the figure, and avoids raw geopolitics. This structurally defuses the failure mode rather than filtering after the fact. It opens with a curated question, then generates each follow-up from the transcript so far — a real adaptive interview, not a questionnaire.

4.4 · Bio-grounded delivery (new)

The identity card now also derives, from curated data only, a `voice_descriptor`: full birth/death dates when ingested, an exact lifespan via signed-year arithmetic (Plato $-428 \rightarrow -348 = 80$ yrs), and the Wikipedia one-line description — which already encodes origin and role (e.g. “German-born theoretical

physicist"). No native language or accent is invented; the model is told to let period and stated origin colour the register, nothing more. The same string is the intended seed for the future text-to-speech voice.

4.5 · Episode composer

Stitches a themed cold-open quote, the host framing, the conversational turn loop (history threaded through every answer for coherence and non-repetition), and a sign-off into JSON + a readable Markdown transcript. A 6-question episode is ≈ 12 Claude calls, $\approx \$0.30$ on Sonnet.

5 · Orchestration & infrastructure

Two Airflow DAGs (LocalExecutor, manual-trigger). `bdm_p2_trusted_zone` cleans the nine sources and, on completion, auto-triggers `bdm_p2_exploitation_zone` via `TriggerDagRunOperator`. The Exploitation DAG gates on MinIO and Milvus health, builds `dim_figure`, then the facts. Fact builds are **serialized with chain()** because DuckDB is single-writer — parallel writers collide on its file lock. `corpus_chunks` then embeds, and two verifier tasks (DuckDB row sanity, per-source Milvus existence) gate 'complete'.

Milvus runs as a standalone Docker stack (milvus + etcd + an internal-only MinIO, kept portless so it cannot collide with the lakehouse MinIO). The embedding deps (CPU torch + sentence-transformers + pymilvus) and the Anthropic SDK are baked into the Airflow image; the consumption package is mounted in so it has the deps and can reach Milvus.

6 · Data products & status

Component	Consumer	Status
<code>dim_figure</code> + 4 facts	Dashboard KPIs, agents	Green
<code>corpus_chunks</code> (Milvus)	Reasoner/Voice RAG	Green (≈87k)
Conversational podcast	End user (demo)	Green, live
Bio voice_descriptor	Text register + future TTS	Done
Streamlit dashboard	Project demo	Planned (#20)
<code>fact_mentions_1m</code>	Real-time KPI	Planned (Santi)
Text-to-speech	Audio episode	Planned

7 · Planned integration — streaming (Santi sync)

Santi's branch runs a Spark Structured Streaming job: it reads the Kafka character-mentions topic, applies a 1-minute tumbling window with a 30-second watermark, aggregates to (window_start, window_end, character_name, domain, mention_count, avg_sentiment) and writes Parquet to /app/trusted/streaming/fact_mentions_1m. It slots into our Exploitation Zone as a real-time fact table joining dim_figure.

Planned Integration — Real-Time Mentions



OPEN SEAM DECISIONS (Santi sync)

1. character_name ↔ figure_slug naming convention
2. Parquet view (read_parquet) vs periodic batch load into exploit.duckdb
3. Branch merge strategy — origin/santi forked pre-Trusted; needs rebase, not fast-forward

Figure 4 — The streaming seam and the three decisions to settle in tomorrow's sync.

Decisions for the meeting:

- **Naming** — his character_name must reconcile with our figure_slug key, or the join silently drops rows.
- **Landing form** — expose the Parquet as a DuckDB read_parquet view (always fresh, zero copy) versus a periodic batch load into exploit.duckdb (stable, queryable offline). Recommendation: a view for the live KPI, materialized on DAG run for reproducibility.
- **Merge** — origin/santi forked *before* our Trusted commits, so its diff shows our work as deleted; integration is a **rebase**, never a fast-forward.

Once joined, fact_mentions_1m gives the dashboard a live “who is being talked about now” signal and could feed the Interviewer a real-time topical cue — closing the loop between the streaming and consumption ends of the pipeline.